SDK Advanced Topics

Advanced patterns and techniques for the Graph OLAP Python SDK.

---

1. IPython Magic Commands

The SDK provides IPython magic commands for streamlined notebook workflows. Load the extension once per session for instant access to graph queries.

Loading the Extension

```
# Load the graph_olap IPython extension
%load_ext graph_olap

# Verify it loaded successfully
%graph_query RETURN 1 AS test
```

Line Magic: Quick Queries

Use `%graph_query` for single-line Cypher queries with immediate results:

```
# Count all nodes
%graph_query MATCH (n) RETURN count(n) AS total_nodes

# Quick lookup by ID
%graph_query MATCH (c:Customer {id: 'C001'}) RETURN c.name, c.email

# Aggregation queries
%graph_query MATCH (c:Customer)-[:PURCHASED]->(p:Product) RETURN c.name,
count(p) AS purchases ORDER BY purchases DESC LIMIT 5
```

Cell Magic: Multi-Line Queries

Use `%%cypher` for complex, multi-line queries with formatting:

```
%%cypher
MATCH (c:Customer)-[p:PURCHASED]->(prod:Product)
WHERE prod.category = 'Electronics'
WITH c, count(p) AS purchases, sum(p.amount) AS total_spent
WHERE purchases >= 3
RETURN c.name AS customer,
       purchases,
       total_spent,
       total_spent / purchases AS avg_order_value
ORDER BY total_spent DESC
LIMIT 10
```

Magic Command Options

Both magic commands support options for output control:

```
# Return as Polars DataFrame (default)
%graph_query --format polars MATCH (n:Customer) RETURN n.name, n.age LIMIT 100

# Return as Pandas DataFrame
%graph_query --format pandas MATCH (n:Customer) RETURN n.name, n.age LIMIT 100

# Return raw QueryResult object
%graph_query --format raw MATCH (n:Customer) RETURN n.name, n.age LIMIT 100

# Set query timeout (milliseconds)
%graph_query --timeout 30000 MATCH (n)-[*1..5]-(m) RETURN count(*) AS paths
```

Cell magic options:

```
%%cypher --format pandas --timeout 60000
MATCH path = shortestPath((a:Customer {id: $source})-[*]-(b:Customer {id:
$target}))
RETURN path
```

Using Parameters

Pass parameters to magic commands using the `--params` option:

```
# Line magic with parameters (JSON format)
%graph_query --params {"city": "London", "min_age": 25} MATCH (c:Customer)
WHERE c.city = $city AND c.age >= $min_age RETURN c.name

# Cell magic with parameters
%%cypher --params {"category": "Electronics", "min_purchases": 5}
MATCH (c:Customer)-[p:PURCHASED]->(prod:Product)
WHERE prod.category = $category
WITH c, count(p) AS purchases
WHERE purchases >= $min_purchases
RETURN c.name, purchases
```

Connection Context

The magic commands use the current notebook connection:

```
from graph_olap import notebook
from graph_olap_schemas import WrapperType

# Establish connection (required before using magics)
client = notebook.connect()
# Create instance directly from mapping (snapshot managed internally)
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=1,
    name="Analysis",
    wrapper_type=WrapperType.RYUGRAPH,
)
conn = client.instances.connect(instance.id)

# Set connection for magics
%graph_connect conn

# Now magics use this connection
%graph_query MATCH (n) RETURN labels(n)[0] AS label, count(*) AS count
```

---

2. Error Handling

The SDK provides a comprehensive exception hierarchy that maps directly to API error responses. All exceptions inherit from `GraphOLAPError`.

Exception Hierarchy

```
GraphOLAPError (base)
|
+-- AuthenticationError
|   Invalid or missing API key (HTTP 401)
|
+-- PermissionDeniedError
|   |   User lacks permission for operation
|   |
|   +-- ForbiddenError
|       Access forbidden, lacks required role (HTTP 403)
|
+-- NotFoundError
|   Resource not found (HTTP 404)
|
+-- ValidationError
|   Request validation failed (HTTP 422)
|
+-- ConflictError
|   |   Operation conflicts with current state (HTTP 409)
|   |
|   +-- ResourceLockedError
|   |   Instance locked by running algorithm
|   |
|   +-- ConcurrencyLimitError
|   |   Instance creation limit exceeded (HTTP 429)
|   |
|   +-- DependencyError
|   |   Resource has dependencies preventing deletion
|   |
|   +-- InvalidStateError
|       Operation invalid for current resource state
|
+-- TimeoutError
|   |   Operation timed out
|   |
|   +-- QueryTimeoutError
|   |   Cypher query exceeded timeout
|   |
|   +-- AlgorithmTimeoutError
|       Algorithm execution exceeded timeout
|
+-- RyugraphError
|   Cypher syntax or execution error
|
+-- AlgorithmNotFoundError
|   Unknown algorithm name
|
+-- AlgorithmFailedError
|   Algorithm execution failed
|
+-- SnapshotFailedError
|   Snapshot export failed
|
```

```
+-- InstanceFailedError
|   Instance startup failed
|
+-- ServerError
    |   Server-side error (HTTP 5xx)
    |
    +-- ServiceUnavailableError
        Service temporarily unavailable (HTTP 503)
```

Basic Exception Handling

```python
from graph_olap import GraphOLAPClient
from graph_olap.exceptions import (
    GraphOLAPError,
    NotFoundError,
    ValidationError,
    ResourceLockedError,
)

client = GraphOLAPClient.from_env()

# Catch specific exceptions
try:
    mapping = client.mappings.get(999)
except NotFoundError as e:
    print(f"Mapping not found: {e}")
    print(f"Details: {e.details}")

# Catch category of exceptions
try:
    instance = client.instances.create_from_mapping_and_wait(
        mapping_id=1,
        name="Test",
        wrapper_type=WrapperType.RYUGRAPH,  # Required: RYUGRAPH or FALKORDB
    )
except ConflictError as e:
    # Handles ResourceLockedError, ConcurrencyLimitError, etc.
    print(f"Conflict error: {e}")

# Catch all SDK errors
try:
    result = conn.query("MATCH (n) RETURN n")
except GraphOLAPError as e:
    print(f"SDK error: {e}")
```

Handling Resource Locks

```python
from graph_olap.exceptions import ResourceLockedError
import time

def run_algorithm_with_retry(conn, max_retries=3, wait_seconds=30):
    """Run algorithm with retry on lock conflict."""
    for attempt in range(max_retries):
        try:
            return conn.algo.pagerank("Customer", "pr_score")
        except ResourceLockedError as e:
            print(f"Instance locked by {e.holder_name} running {e.algorithm}")
            if attempt < max_retries - 1:
                print(f"Waiting {wait_seconds}s before retry...")
                time.sleep(wait_seconds)
            else:
                raise

# Check lock status before running
lock = conn.get_lock()
if lock.locked:
    print(f"Instance locked until algorithm completes")
    print(f"  Holder: {lock.holder_name}")
    print(f"  Algorithm: {lock.algorithm}")
    print(f"  Acquired: {lock.acquired_at}")
else:
    exec_result = conn.algo.pagerank("Customer", "pr_score")
```

Handling Concurrency Limits

```python
from graph_olap.exceptions import ConcurrencyLimitError
from graph_olap_schemas import WrapperType

try:
    instance = client.instances.create_from_mapping_and_wait(
        mapping_id=mapping.id,
        name="Analysis Instance",
        wrapper_type=WrapperType.RYUGRAPH,  # Required: RYUGRAPH or FALKORDB
    )
except ConcurrencyLimitError as e:
    print(f"Limit type: {e.limit_type}")
    print(f"Current count: {e.current_count}")
    print(f"Maximum allowed: {e.max_allowed}")

    # List existing instances to find one to terminate
    instances = client.instances.list(status="running")
    print(f"You have {len(instances.items)} running instances")
```

Handling Query Errors

```
from graph_olap.exceptions import RyugraphError, QueryTimeoutError

try:
    result = conn.query(
        "MATCH (n)-[*1..10]-(m) RETURN count(*)",
        timeout=30.0,
    )
except QueryTimeoutError:
    print("Query timed out. Try:")
    print("  - Adding LIMIT clause")
    print("  - Reducing path length")
    print("  - Filtering by node labels")
except RyugraphError as e:
    print(f"Cypher error: {e}")
    print(f"Details: {e.details}")  # May include syntax location
```

Retry Patterns with Tenacity

The SDK uses tenacity for internal retries. You can use it for your own retry logic:

```
from tenacity import (
    retry,
    retry_if_exception_type,
    stop_after_attempt,
    wait_exponential,
    before_sleep_log,
)
import logging

logger = logging.getLogger(__name__)

@retry(
    retry=retry_if_exception_type(ResourceLockedError),
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=10, min=10, max=120),
    before_sleep=before_sleep_log(logger, logging.WARNING),
    reraise=True,
)
def run_pagerank(conn):
    """Run PageRank with automatic retry on lock."""
    return conn.algo.pagerank("Customer", "pr_score")

# Retry on transient server errors
@retry(
    retry=retry_if_exception_type(ServiceUnavailableError),
    stop=stop_after_attempt(3),
```

```
    wait=wait_exponential(multiplier=1, min=2, max=30),
    reraise=True,
)
def get_mapping_with_retry(client, mapping_id):
    """Get mapping with retry on 503."""
    return client.mappings.get(mapping_id)
```

Context Manager for Error Handling

```
from contextlib import contextmanager
from graph_olap.exceptions import GraphOLAPError

@contextmanager
def safe_graph_operation(operation_name):
    """Context manager for safe graph operations with logging."""
    try:
        yield
    except NotFoundError as e:
        logger.warning(f"{operation_name}: Resource not found - {e}")
        raise
    except ValidationError as e:
        logger.error(f"{operation_name}: Validation failed - {e}")
        logger.error(f"Details: {e.details}")
        raise
    except GraphOLAPError as e:
        logger.error(f"{operation_name}: SDK error - {e}")
        raise

# Usage
with safe_graph_operation("Create instance"):
    instance = client.instances.create_from_mapping_and_wait(
        mapping_id=mapping.id,
        name="Analysis Instance",
        wrapper_type=WrapperType.RYUGRAPH,
    )
```

---

3. Performance Tips

Query Optimization

Always use LIMIT for exploratory queries:

```python
# BAD: Returns all nodes (potentially millions)
result = conn.query("MATCH (n:Customer) RETURN n")

# GOOD: Limit results for exploration
result = conn.query("MATCH (n:Customer) RETURN n LIMIT 100")

# BETTER: Count first, then decide
count = conn.query_scalar("MATCH (n:Customer) RETURN count(n)")
print(f"Total customers: {count}")

if count < 10000:
    result = conn.query("MATCH (n:Customer) RETURN n")
```

Use projections instead of returning full nodes:

```python
# BAD: Returns entire node with all properties
result = conn.query("MATCH (c:Customer) RETURN c LIMIT 1000")

# GOOD: Return only needed properties
result = conn.query("""
    MATCH (c:Customer)
    RETURN c.id, c.name, c.email
    LIMIT 1000
""")

# Memory savings can be 10x or more for property-rich nodes
```

Use parameterized queries for repeated patterns:

```python
# BAD: String interpolation (no query plan caching, SQL injection risk)
for city in cities:
    result = conn.query(f"MATCH (c:Customer {{city: '{city}'}}) RETURN c")

# GOOD: Parameterized query (plan caching, safe)
for city in cities:
    result = conn.query(
        "MATCH (c:Customer {city: $city}) RETURN c",
        parameters={"city": city}
    )
```

Filter early in the query:

```
# BAD: Filter after pattern matching
result = conn.query("""
    MATCH (c:Customer)-[p:PURCHASED]->(prod:Product)
    WITH c, prod, p
    WHERE prod.category = 'Electronics'
    RETURN c.name, prod.name, p.amount
""")

# GOOD: Filter inline during matching
result = conn.query("""
    MATCH (c:Customer)-[p:PURCHASED]->(prod:Product {category: 'Electronics'})
    RETURN c.name, prod.name, p.amount
""")
```

Connection Pooling and Reuse

Reuse client and connection objects:

```
# BAD: Creating new client for each operation
for mapping_id in mapping_ids:
    client = GraphOLAPClient.from_env()
    mapping = client.mappings.get(mapping_id)
    client.close()

# GOOD: Reuse client
client = GraphOLAPClient.from_env()
try:
    for mapping_id in mapping_ids:
        mapping = client.mappings.get(mapping_id)
finally:
    client.close()

# BEST: Use context manager
with GraphOLAPClient.from_env() as client:
    for mapping_id in mapping_ids:
        mapping = client.mappings.get(mapping_id)
```

Reuse instance connections:

```
# BAD: Reconnecting for each query
for query in queries:
    conn = client.instances.connect(instance_id)
    result = conn.query(query)
    conn.close()
```

```python
# GOOD: Reuse connection
conn = client.instances.connect(instance_id)
try:
    for query in queries:
        result = conn.query(query)
finally:
    conn.close()

# Using context manager
with client.instances.connect(instance_id) as conn:
    for query in queries:
        result = conn.query(query)
```

Algorithm Efficiency

Specify node labels to reduce scope:

```python
# BAD: Operates on all nodes
exec = conn.algo.pagerank(node_label=None, property_name="pr")

# GOOD: Operate on specific label
exec = conn.algo.pagerank(node_label="Customer", property_name="pr")
```

Use edge_type to filter relationships:

```python
# Process only PURCHASED relationships for customer analysis
exec = conn.algo.pagerank(
    node_label="Customer",
    property_name="purchase_influence",
    edge_type="PURCHASED"
)
```

Batch related algorithm executions:

```python
# Run complementary algorithms in sequence on same subgraph
algorithms = [
    ("pagerank", {"node_label": "Customer", "property_name": "pr_score"}),
    ("louvain", {"node_label": "Customer", "property_name": "community"}),
    ("connected_components", {"node_label": "Customer", "property_name":
"component"}),
]

for algo_name, params in algorithms:
```

```
    exec = conn.algo.run(algo_name, **params)
    print(f"{algo_name}: Updated {exec.nodes_updated} nodes")
```

DataFrame Format Selection

Choose Polars for large datasets:

```
# Polars is faster and more memory-efficient for large results
df = conn.query_df(
    "MATCH (c:Customer) RETURN c.id, c.name, c.age",
    backend="polars"  # Default
)

# Fast filtering without copying data
filtered = df.filter(pl.col("age") > 25)

# Lazy evaluation for complex pipelines
lazy_df = df.lazy().filter(pl.col("age") > 25).group_by("age").agg(pl.count())
result = lazy_df.collect()
```

Use Pandas when needed for compatibility:

```
# Pandas for libraries that require it (sklearn, matplotlib, etc.)
df = conn.query_df(
    "MATCH (c:Customer) RETURN c.id, c.name, c.age",
    backend="pandas"
)

# Works with sklearn
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df["age_scaled"] = scaler.fit_transform(df[["age"]])
```

Convert between formats efficiently:

```
# Start with Polars for processing
df_polars = conn.query_df("MATCH (n) RETURN n.id, n.value")

# Convert to Pandas only when needed
df_pandas = df_polars.to_pandas()
```

```
# Or use Arrow for zero-copy when possible
arrow_table = df_polars.to_arrow()
```

Memory Management

Process large results in chunks:

```
# Get total count first
total = conn.query_scalar("MATCH (n:Transaction) RETURN count(n)")

# Process in chunks
chunk_size = 10000
for offset in range(0, total, chunk_size):
    result = conn.query(f"""
        MATCH (t:Transaction)
        RETURN t.id, t.amount, t.date
        ORDER BY t.id
        SKIP {offset}
        LIMIT {chunk_size}
    """)

    df = result.to_polars()
    # Process chunk...
    del df  # Free memory
```

Use streaming for very large exports:

```
# Query with YIELD for streaming (if supported)
# Otherwise, use chunked processing as above
```

---

4. Troubleshooting

Common Issues and Solutions

Issue	Cause	Solution
`ConnectionError: Connection refused`	Instance not ready	Call `wait_until_running()` before `connect()`

Issue	Cause	Solution
`ResourceLockedError`	Algorithm running	Check `conn.get_lock()`, wait for completion
`QueryTimeoutError`	Query too complex	Add `LIMIT`, optimize query, increase timeout
`ConcurrencyLimitError`	Too many instances	Terminate unused instances
`InvalidStateError`	Wrong resource state	Check status before operations
`NotFoundError`	Resource deleted	Verify resource exists
`RyugraphError`	Cypher syntax error	Check query syntax
`AuthenticationError`	Invalid API key	Verify `GRAPH_OLAP_API_KEY`
`PermissionDeniedError`	Insufficient permissions	Check user role
`SnapshotFailedError`	Internal export failed	Check mapping SQL, Starburst connectivity
`InstanceFailedError`	Pod startup failed	Check snapshot status, cluster resources

Connection Issues

Instance not ready:

```
from graph_olap_schemas import WrapperType

# Problem: Connection refused if you don't wait for instance to be ready
instance = client.instances.create_from_mapping(
    mapping_id=1,
    name="Test",
    wrapper_type=WrapperType.RYUGRAPH,  # Required: RYUGRAPH or FALKORDB
```

```
)
conn = client.instances.connect(instance.id)  # Fails! (instance still
starting)

# Solution: Use create_from_mapping_and_wait which waits for ready state
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=1,
    name="Test",
    wrapper_type=WrapperType.RYUGRAPH,  # Required: RYUGRAPH or FALKORDB
    timeout=900,  # 15 minutes max (includes snapshot export time)
)
conn = client.instances.connect(instance.id)  # Works!
```

Instance shows running but connection fails:

```
from graph_olap.exceptions import InvalidStateError

# Check instance health
try:
    health = client.instances.get_health(instance.id)
    print(f"Health status: {health}")
except ConnectionError as e:
    print(f"Instance not reachable: {e}")

    # May need to wait longer for network setup
    import time
    time.sleep(10)

    # Retry
    health = client.instances.get_health(instance.id)
```

Algorithm Issues

Algorithm lock conflicts:

```
# Check current lock before running algorithm
lock = conn.get_lock()
if lock.locked:
    print(f"Instance locked by: {lock.holder_name}")
    print(f"Running algorithm: {lock.algorithm}")
    print(f"Started at: {lock.acquired_at}")

    # Option 1: Wait for completion
    import time
    while conn.get_lock().locked:
        time.sleep(5)
```

```
    # Option 2: Use a different instance
    print("Consider using a different instance")
```

Algorithm timeout:

```
from graph_olap.exceptions import AlgorithmTimeoutError

try:
    exec = conn.algo.pagerank(
        "Customer",
        "pr_score",
        timeout=600,  # 10 minutes
    )
except AlgorithmTimeoutError:
    # Algorithm may still be running
    # Check status endpoint
    status = conn.algo.get_status(exec.execution_id)
    print(f"Algorithm status: {status}")
```

Query Issues

Cypher syntax errors:

```
from graph_olap.exceptions import RyugraphError

try:
    result = conn.query("MATCH (n:Customer RETURN n")  # Missing )
except RyugraphError as e:
    print(f"Cypher error: {e}")
    # Check e.details for position information
```

Query returns unexpected results:

```
# Debug by checking schema first
schema = conn.get_schema()
print("Node labels:", list(schema.node_labels.keys()))
print("Relationship types:", list(schema.relationship_types.keys()))

# Check if label exists
if "Customer" not in schema.node_labels:
    print("WARNING: Customer label not in graph!")
```

```
# Verify property names
customer_props = schema.node_labels.get("Customer", {})
print("Customer properties:", customer_props)
```

Instance Creation Issues

Instance stuck in waiting_for_snapshot:

```
from graph_olap import notebook
from graph_olap_schemas import WrapperType

# Wake Starburst cluster if using Galaxy
notebook.wake_starburst(timeout=120)

# Create instance directly from mapping (snapshot managed internally)
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=mapping.id,
    name="My Instance",
    wrapper_type=WrapperType.RYUGRAPH,
    timeout=600,
)
```

Instance creation failed (internal snapshot failed):

```
from graph_olap.exceptions import InstanceFailedError
from graph_olap_schemas import WrapperType

try:
    instance = client.instances.create_from_mapping_and_wait(
        mapping_id=mapping.id,
        name="Test Instance",
        wrapper_type=WrapperType.RYUGRAPH,
    )
except InstanceFailedError as e:
    # Get instance details for error message
    instance = client.instances.get(instance_id)
    print(f"Failed: {instance.error_message}")

    # Common causes:
    # - Invalid SQL in mapping
    # - Starburst connectivity issues
    # - Missing tables or columns
```

Environment Issues

Missing environment variables:

```
import os

required_vars = [
    "GRAPH_OLAP_API_URL",
    "GRAPH_OLAP_API_KEY",
]

missing = [v for v in required_vars if not os.environ.get(v)]
if missing:
    print(f"Missing environment variables: {missing}")
```

---

5. Logging and Debugging

Enable Debug Logging

The SDK uses Python's standard logging module:

```
import logging

# Enable debug logging for all SDK modules
logging.basicConfig(level=logging.DEBUG)

# Or configure specific loggers
logging.getLogger("graph_olap").setLevel(logging.DEBUG)
logging.getLogger("graph_olap.http").setLevel(logging.DEBUG)
logging.getLogger("httpx").setLevel(logging.DEBUG)  # HTTP request details
```

Structured Logging Configuration

```
import logging
import sys

# Create formatter with timestamp and module
formatter = logging.Formatter(
    "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
)

# Console handler
```

```
console_handler = logging.StreamHandler(sys.stdout)
console_handler.setFormatter(formatter)

# File handler for persistent logs
file_handler = logging.FileHandler("graph_olap.log")
file_handler.setFormatter(formatter)

# Configure SDK logger
logger = logging.getLogger("graph_olap")
logger.setLevel(logging.DEBUG)
logger.addHandler(console_handler)
logger.addHandler(file_handler)
```

Request Tracing

Track requests with unique IDs for debugging:

```
import uuid
import logging

logger = logging.getLogger(__name__)

def traced_query(conn, cypher, parameters=None):
    """Execute query with request tracing."""
    request_id = str(uuid.uuid4())[:8]

    logger.info(f"[{request_id}] Executing query: {cypher[:100]}...")
    if parameters:
        logger.debug(f"[{request_id}] Parameters: {parameters}")

    try:
        result = conn.query(cypher, parameters)
        logger.info(f"[{request_id}] Query completed: {result.row_count} rows")
        return result
    except Exception as e:
        logger.error(f"[{request_id}] Query failed: {e}")
        raise

# Usage
result = traced_query(conn, "MATCH (n:Customer) RETURN n LIMIT 10")
```

Debugging HTTP Requests

```
import logging
import httpx
```

```python
# Enable httpx logging for detailed request/response info
logging.getLogger("httpx").setLevel(logging.DEBUG)

# Or use custom event hooks
def log_request(request):
    print(f"Request: {request.method} {request.url}")
    print(f"Headers: {dict(request.headers)}")

def log_response(response):
    print(f"Response: {response.status_code}")
    print(f"Body preview: {response.text[:500]}...")

# Note: For production, use structured logging instead of print
```

Performance Profiling

```python
import time
from functools import wraps

def timed(func):
    """Decorator to time function execution."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.3f}s")
        return result
    return wrapper

# Usage
@timed
def analyze_customers(conn):
    result = conn.query("MATCH (c:Customer) RETURN c LIMIT 1000")
    df = result.to_polars()
    return df.group_by("city").agg(pl.count())

# Or inline timing
start = time.perf_counter()
result = conn.query("MATCH (n)-[r]->(m) RETURN count(*)")
print(f"Query took {time.perf_counter() - start:.3f}s")
```

Debug Mode for Testing

```python
from graph_olap import notebook
from graph_olap.testing import TestPersona
```

```python
# Enable verbose output for test context
import logging
logging.getLogger("graph_olap.testing").setLevel(logging.DEBUG)

# Create test context with debug info
ctx = notebook.test("DebugTest", persona=TestPersona.ANALYST_ALICE)

# Resources are tracked and logged
mapping = ctx.mapping(node_definitions=[...])  # Logs: "Tracking mapping 123
(DebugTest-Mapping-abc123)"

# Cleanup shows what was cleaned
results = ctx.cleanup()  # Logs cleanup actions
print(f"Cleaned: {results}")
```

Inspecting SDK Objects

```python
# Inspect instance details
instance = client.instances.get(123)
print(f"Instance ID: {instance.id}")
print(f"Name: {instance.name}")
print(f"Status: {instance.status}")
print(f"URL: {instance.instance_url}")
print(f"Created: {instance.created_at}")
print(f"Expires: {instance.expires_at}")

# Inspect query result metadata
result = conn.query("MATCH (n) RETURN n LIMIT 10")
print(f"Columns: {result.columns}")
print(f"Column types: {result.column_types}")
print(f"Row count: {result.row_count}")
print(f"Execution time: {result.execution_time_ms}ms")

# Inspect algorithm execution
exec = conn.algo.pagerank("Customer", "pr_score")
print(f"Execution ID: {exec.execution_id}")
print(f"Status: {exec.status}")
print(f"Nodes updated: {exec.nodes_updated}")
print(f"Duration: {exec.duration_ms}ms")
```

Common Debug Patterns

Verify connection is working:

```python
def verify_connection(conn):
    """Verify connection with diagnostic info."""
    try:
        # Test basic query
        result = conn.query("RETURN 1 AS test")
        assert result.scalar() == 1
        print("Basic query: OK")

        # Test schema access
        schema = conn.get_schema()
        print(f"Schema: {len(schema.node_labels)} node labels")

        # Test lock status
        lock = conn.get_lock()
        print(f"Lock status: {'locked' if lock.locked else 'unlocked'}")

        # Test instance status
        status = conn.status()
        print(f"Memory usage: {status.get('memory_usage', 'N/A')}")

        return True
    except Exception as e:
        print(f"Connection verification failed: {e}")
        return False

# Usage
if verify_connection(conn):
    print("Connection verified successfully")
```

Debug query results:

```python
def debug_query(conn, cypher, parameters=None):
    """Execute query with full debugging output."""
    print(f"Query: {cypher}")
    if parameters:
        print(f"Parameters: {parameters}")

    result = conn.query(cypher, parameters)

    print(f"\nColumns ({len(result.columns)}):")
    for i, (col, dtype) in enumerate(zip(result.columns, result.column_types)):
        print(f"  {i}: {col} ({dtype})")

    print(f"\nRows: {result.row_count}")
    if result.row_count > 0:
        print("\nFirst row:")
        for col, val in zip(result.columns, result.rows[0]):
            print(f"  {col}: {val!r}")
```

```
    print(f"\nExecution time: {result.execution_time_ms}ms")

    return result
```